Seminar 02

16.10.2026

Lernziel: HTML, CSS und JavaScript Grundlagen.

1. HTML

1. Erstelle eine Seminarseite mit:
   - Einer Überschrift
   - Einem Absatz
   - Ein Bild
2. Baue ein Kontaktformular mit Feldern für Name, E-Mail und Nachricht (achte auf korrekte `label`-Verwendung)
3. Erstelle eine Tabelle mit dem Seminarplan (Zeit, Thema, Raum) mit `thead`, `tbody`

2. CSS

1. Style die Seminarseite: Schiftart, Textfarbe, Hintergrund und ein zentriertes Layout mit `max-width`
2. Gib allen Überschriften im `main`-Bereich eine blaue Farbe, ohne Klassen zu verwenden
3. Baue eine Karten-Reihe (drei nebeneinanderliegende Boxen) mit Flexbox, die auf Mobilgeräten untereinander stapelt (`@media`)
4. Erstelle ein 2-Spalten-Layout (Sidebar + Inhalt) mit CSS Grid: `grid-template-area`, mit responsivem Umbruch auf eine Spalte

3. JavaScript

1. Schreibe eine Funktion `greeting(name)`, die `"Hello, [name]!"` zurückgibt und das Ergebnis per `console.log` ausgibt.
2. Ein Button soll bei jedem Klick einen Zähler um 1 erhöhen und die aktuelle Zahl auf der Seite anzeigen (DOM-Manipulation)
3. Validiere das Kontaktformular aus Aufgabe 1.2 per JavaScript: Name und E-Mail dürfen nicht leer sein, die Email muss ein `@` enthalten. Zeige Fehlermeldungen unter den Feldern
4. Schreibe eine Funktion `average(number)`, die den Mittelwert eines Arrays berechnet und auf 2 Nachkommastellen rundet.
5. Erstelle eine Todo-Liste
   - Einträge hinzufügen
   - Abhaken von Einträgen
   - Löschen
   - Liste soll bei Reload mit `localStorage` erhalten bleiben

4. JavaScript Cheatsheet

4.1. Variables and Types

```JavaScript
const x = 1;          // block-scoped, no reassignment
let y = 2;            // block-scoped, reassignable
// avoid var (function-scoped, hoisted)

typeof 42             // "number"
typeof "hi"           // "string"
typeof null           // "object" (historic quirk)
Array.isArray([])     // true
```

4.2. Equality and Truthiness

```JavaScript
 1 === 1      // strict, use this by default
 1 == "1"     // loose, coerces types

// Falsy: false, 0, -0, 0n, "", null, undefined, NaN
// Everything else is truthy (including [], {}, "0")

a ?? b       // b only if a is null/undefined
a || b       // b if a is falsy
a?.b?.[0]    // optional chaining
a ??= 5      // assign if null/undefined
```

4.3. Strings

```JavaScript
const s = `Hello ${name}`;       // template literal
s.length; s.at(-1);
s.includes("a"); s.startsWith("H"); s.endsWith("o");
s.slice(1, 3); s.split(","); s.trim();
s.replace("a", "b"); s.replaceAll("a", "b");
s.padStart(5, "0"); s.toUpperCase();
```

4.4. Arrays

```JavaScript
const a = [1, 2, 3];
a.push(4); a.pop(); a.unshift(0); a.shift();

a.map(n => n * 2);
a.filter(n => n > 1);
a.reduce((sum, n) => sum + n, 0);
a.find(n => n > 1);        a.findIndex(n => n > 1);
a.some(n => n > 2);        a.every(n => n > 0);
a.includes(2);             a.indexOf(2);
a.flat();                  a.flatMap(n => [n, n]);
a.slice(1, 3);             // non-mutating
a.splice(1, 1);            // mutating
```

```
[...a].sort((x, y) => x - y);   // copy then sort
a.toSorted(); a.toReversed();   // non-mutating (ES2023)
a.at(-1);
Array.from({ length: 3 }, (_, i) => i);   // [0, 1, 2]
```

4.5. Objects

```javascript
const o = { a: 1, b: 2 };
o.a; o["b"];
const { a, b: renamed = 0, ...rest } = o;   // destructuring
const copy = { ...o, c: 3 };                // spread

Object.keys(o); Object.values(o); Object.entries(o);
Object.fromEntries([["a", 1]]);
Object.assign({}, o);
structuredClone(o);          // deep copy
"a" in o; delete o.a;
```

4.6. Functions

```javascript
function add(a, b = 0) { return a + b; }
const add2 = (a, b) => a + b;
const sum = (...nums) => nums.reduce((s, n) => s + n, 0);
add(...[1, 2]);

// Closures
const counter = () => { let c = 0; return () => ++c; };
```

Arrow functions have no own `this`, `arguments`, or `new`.

4.7. Control flow

```javascript
if (x) {} else if (y) {} else {}
cond ? a : b;
switch (v) { case 1: break; default: }

for (let i = 0; i < n; i++) {}
for (const item of iterable) {}   // values
for (const key in obj) {}         // keys (avoid on arrays)
while (cond) {}
```

4.8. Classes

```javascript
class Animal {
  #secret = 1;                    // private field
  static count = 0;
  constructor(name) { this.name = name; }
  speak() { return `${this.name} makes a sound`; }
  get upper() { return this.name.toUpperCase(); }
```

```
7   }
8   class Dog extends Animal {
9     speak() { return super.speak() + " (woof)"; }
10  }
```

4.9. Modules

```
1  export const x = 1;
2  export default function main() {}
3  import main, { x } from "./file.js";
4  import * as utils from "./utils.js";
5  const mod = await import("./lazy.js");   // dynamic
```

4.10. Async

```
1   const wait = ms => new Promise(r => setTimeout(r, ms));
2
3   async function load() {
4     try {
5       const res = await fetch("/api");
6       if (!res.ok) throw new Error(res.status);
7       return await res.json();
8     } catch (e) {
9       console.error(e);
10    } finally {
11      // cleanup
12    }
13  }
14
15  await Promise.all([p1, p2]);        // fails fast
16  await Promise.allSettled([p1, p2]); // never rejects
17  await Promise.race([p1, p2]);
18  await Promise.any([p1, p2]);        // first success
```

Event loop order: sync code, then microtasks (promises), then macrotasks (`setTimeout`).

4.11. Map, Set, and friends

```
1  const m = new Map([["k", 1]]);
2  m.set("k2", 2); m.get("k"); m.has("k"); m.delete("k"); m.size;
3  const s = new Set([1, 1, 2]);   // unique values
4  [...new Set(arr)];              // dedupe
5  new WeakMap(); new WeakSet();
```

4.12. DOM basics

```
1  const el = document.querySelector(".cls");
2  document.querySelectorAll("li");
3  el.textContent = "hi";
```

```
el.classList.add("on"); el.classList.toggle("on");
el.setAttribute("data-x", "1"); el.dataset.x;
el.addEventListener("click", e => { e.preventDefault(); });
parent.append(child); el.remove();
```

4.13. Useful built-ins

```
JSON.stringify(obj, null, 2); JSON.parse(str);
Math.max(...a); Math.round(x); Math.floor(x); Math.random();
Number("42"); parseInt("42", 10); parseFloat("1.5");
(0.1 + 0.2).toFixed(2);
new Date().toISOString();
setTimeout(fn, 100); setInterval(fn, 100);
new URL("https://a.com?q=1").searchParams.get("q");
```